

Full Stack Dev

Interview Notes

Syslab.ai · Aptitude + Technical + Code Snippets

■ Table of Contents

1.

HTML / CSS — The Bread & Butter

Front-end foundation
2.

JavaScript — The Espresso Shot

Core language
3.

React.js — The Latte Art

UI framework
4.

Node.js & Express — The Kitchen Engine

Back-end
5.

REST APIs & HTTP — The Menu Card

API design
6.

Databases: SQL & NoSQL — The Pantry

Data layer
7.

Git & Version Control — The Recipe Book

Dev workflow
8.

AI/ML & Syslab Stack — The Secret Sauce

Company-specific
9.

System Design — The Chef's Blueprint

Architecture
- 1

Aptitude & Logic — The Taste Test

Problem solving
0.

Code Snippets Bank — The Cookbook

Practice code

■ 1. HTML / CSS — The Bread & Butter

Core Concepts

HTML (HyperText Markup Language) defines the structure of web pages using elements/tags. CSS (Cascading Style Sheets) controls visual presentation. Together they form the foundation of every web application.

- **DOM (Document Object Model)** — Tree representation of an HTML page. JS can query/modify it.
- **Semantic HTML** — Tags like <header>, <nav>, <main>, <article> for accessibility & SEO.
- **Box Model** — content → padding → border → margin. **box-sizing: border-box** makes sizing intuitive.
- **Flexbox** — 1D layout. **display:flex + justify-content + align-items**.
- **CSS Grid** — 2D layout. **grid-template-columns, grid-template-rows**.
- **Responsive Design** — Media queries: **@media (max-width: 768px) { ... }**
- **CSS Specificity** — inline (1000) > id (100) > class (10) > element (1).
- **Position values** — static / relative / absolute / fixed / sticky.

Likely Interview Questions

Q: What is the difference between id and class?

A: id is unique per page (one element), class can be reused on multiple elements. id has higher CSS specificity.

Q: Explain the CSS box model.

A: Every element is a rectangular box: content area, surrounded by padding (inside border), then border, then margin (outside). **box-sizing:border-box** includes padding/border in the declared width.

Q: What is the difference between display:block, inline, and inline-block?

A: block starts on a new line and takes full width. inline flows in text, ignores width/height. inline-block flows in text but respects width/height.

Q: What are pseudo-classes and pseudo-elements?

A: :hover, :focus, :nth-child() are pseudo-classes (element state). ::before, ::after are pseudo-elements (virtual DOM nodes for decoration).

Q: How does Flexbox differ from CSS Grid?

A: Flexbox is one-dimensional (row or column). Grid is two-dimensional (rows AND columns). Use Flex for component-level layout, Grid for page-level layout.

■ 2. JavaScript — The Espresso Shot

Must-Know Concepts

- **var / let / const** — var is function-scoped & hoisted; let/const are block-scoped. const prevents reassignment.
- **Hoisting** — Declarations are moved to the top of scope. var hoists value as undefined; functions hoist with body.
- **Closures** — A function that remembers its outer lexical scope even after that scope has returned.
- **Event Loop** — JS is single-threaded. Call stack executes sync code; async callbacks go to task queue; event loop pushes them to stack when stack is empty.
- **Promises** — Object representing eventual completion/failure. States: pending → fulfilled | rejected.
- **async/await** — Syntactic sugar over Promises. Makes async code look synchronous.
- **this keyword** — In regular functions, 'this' depends on call site. Arrow functions inherit 'this' from enclosing scope.
- **Prototype Chain** — JS objects inherit from prototypes. ES6 class is syntactic sugar over prototype-based inheritance.
- **Spread / Rest** — ... spreads iterable elements or collects remaining params.
- **Destructuring** — `const {a, b} = obj;` / `const [x, y] = arr;`
- **Map / Filter / Reduce** — Higher-order array methods. Core to functional JS.
- **Debounce vs Throttle** — Debounce delays execution until N ms after last call. Throttle executes at most once every N ms.

Common Q&A;

Q: What is the difference between == and ===?

A: == performs type coercion (1 == '1' is true). === is strict equality with no coercion (1 === '1' is false). Always prefer ===.

Q: Explain event bubbling and capturing.

A: Bubbling: event fires on target, then propagates UP to ancestors. Capturing: fires top-down. addEventListener 3rd arg: false=bubble (default), true=capture. stopPropagation() halts it.

Q: What is a Promise and how does async/await work?

A: A Promise wraps async operations. .then()/.catch() chain callbacks. async/await lets you write async code linearly; await pauses execution until the Promise resolves.

Q: What is the difference between null and undefined?

A: undefined means a variable was declared but not assigned. null is an intentional absence of value (explicitly set).

Q: How does prototypal inheritance work?

A: Each object has a [[Prototype]] link. Property lookup traverses the chain. Object.create(proto) creates an object inheriting from proto. ES6 class syntax wraps this pattern.

■ 3. React.js — The Latte Art

Virtual DOM

React maintains a lightweight in-memory copy of the real DOM. On state change it re-renders into the virtual DOM, diffs with previous version (reconciliation), then updates only changed real DOM nodes (patching). This avoids costly full re-renders.

Core Hooks

- **useState(initial)** — Returns [state, setter]. Triggers re-render on change.
- **useEffect(fn, deps)** — Side effects (fetch, subscriptions). Runs after render. [] = once on mount. Return cleanup fn to avoid leaks.
- **useContext(Ctx)** — Consumes a context value. Avoids prop drilling.
- **useRef(initial)** — Mutable ref that persists across renders. Access DOM nodes or store mutable values without re-rendering.
- **useMemo(fn, deps)** — Memoizes expensive computed value.
- **useCallback(fn, deps)** — Memoizes function reference to prevent child re-renders.
- **useReducer(reducer, init)** — Complex state logic. Like useState but with action dispatch pattern.

Key Concepts Q&A;

Q: What is the difference between state and props?

A: Props are read-only inputs passed from parent to child. State is internal mutable data owned by the component. State changes trigger re-renders; props changes come from parent.

Q: What is prop drilling and how do you solve it?

A: Passing props through many intermediate components. Solved by Context API (global state without external library) or state management (Redux/Zustand).

Q: When would you use useEffect?

A: Fetching data on mount, setting up event listeners/subscriptions, syncing with external systems (localStorage, timers). Always include correct dependencies to avoid stale closures.

Q: What is React.memo?

A: HOC that memoizes a functional component — skips re-render if props are shallowly equal. Use for expensive components whose props rarely change.

Q: Explain the React component lifecycle (functional).

A: Mount: render → DOM insert → useEffect runs. Update: re-render → DOM patch → useEffect (if deps changed). Unmount: cleanup function from useEffect runs.

■ 4. Node.js & Express — The Kitchen Engine

Node.js is a V8-based JavaScript runtime for server-side code. It uses a non-blocking, event-driven I/O model, making it efficient for I/O-heavy applications (APIs, real-time apps, microservices).

Node.js Internals

- **Event Loop Phases** — timers → pending callbacks → idle/prepare → poll → check (setImmediate) → close callbacks.
- **libuv** — C library powering Node's event loop and async I/O with a thread pool for OS tasks.
- **Streams** — Readable, Writable, Duplex, Transform. Process data chunk-by-chunk for memory efficiency (large file uploads/downloads).
- **Cluster module** — Fork child processes to utilise multi-core CPUs. Each worker shares same server port.
- **Worker Threads** — True parallelism for CPU-bound tasks (image processing, ML inference).

Express.js

- **Middleware** — Functions with (req, res, next) signature. Can modify req/res, end request, or pass to next().
- **Router** — `express.Router()` for modular route grouping.
- **Error Middleware** — 4-arg (err, req, res, next) — must be defined last.
- **CORS** — cors package or manual headers to allow cross-origin requests.

Q: What is the Event Loop in Node.js?

A: Single-threaded mechanism that processes async callbacks. When async ops complete (I/O, timers), their callbacks are queued. Event loop picks them up when the call stack is empty.

Q: Difference between `process.nextTick()` and `setImmediate()`?

A: `nextTick()` fires before the next event loop iteration (microtask). `setImmediate()` fires in the check phase of the current or next iteration.

Q: How do you handle errors in Express?

A: Try/catch in async route handlers, then pass error to `next(err)`. Define a global error-handling middleware as the last middleware with 4 arguments.

■ 5. REST APIs & HTTP — The Menu Card

HTTP Methods

Method	Purpose
GET	Retrieve resource. Idempotent. No body.
POST	Create resource. Not idempotent.
PUT	Replace resource entirely. Idempotent.
PATCH	Partial update. Not necessarily idempotent.
DELETE	Remove resource. Idempotent.

HTTP Status Codes

Range	Type	Examples
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Q: What is REST? What are its constraints?

A: Representational State Transfer. Constraints: client-server, stateless (no session on server), cacheable, uniform interface, layered system, optional code-on-demand.

Q: What is the difference between REST and GraphQL?

A: REST uses multiple fixed endpoints (one per resource). GraphQL uses one endpoint; clients specify exactly what data they need, eliminating over-fetching/under-fetching.

Q: What is JWT?

A: JSON Web Token. Header.Payload.Signature (Base64+HMAC). Stateless auth: server signs token, client sends it in Authorization header. Server verifies signature without DB lookup.

Q: What is CORS?

A: Cross-Origin Resource Sharing. Browser security that blocks cross-origin requests by default. Server must send Access-Control-Allow-Origin header to permit specific origins.

■ 6. Databases: SQL & NoSQL — The Pantry

SQL (Relational)

Structured data with defined schema. ACID compliant. Best for complex queries, relationships, and transactional systems.

- **ACID** — Atomicity, Consistency, Isolation, Durability.
- **Normalization** — Organising data to reduce redundancy. 1NF/2NF/3NF.
- **JOINS** — INNER (matching rows both tables), LEFT (all left + matching right), RIGHT, FULL OUTER.
- **Indexing** — B-Tree index speeds up SELECT queries at cost of INSERT/UPDATE overhead.
- **Transactions** — BEGIN / COMMIT / ROLLBACK for atomic multi-statement operations.
- **ORM** — Object-Relational Mapper (Sequelize, Prisma, TypeORM) maps JS objects to DB rows.

NoSQL (MongoDB)

Schema-less documents (JSON/BSON). Horizontally scalable. Best for unstructured/semi-structured data.

- **Collections** = tables, **Documents** = rows (JSON objects).
- **Mongoose** — ODM for Node. Defines schemas & models.
- **Aggregation Pipeline** — \$match → \$group → \$sort → \$project for complex queries.
- **CAP Theorem** — Distributed systems can guarantee only 2 of 3: Consistency, Availability, Partition Tolerance. MongoDB chooses CP.

Q: When to use SQL vs NoSQL?

A: SQL for relational data, complex queries, ACID transactions (banking, ERP). NoSQL for flexible schemas, large-scale horizontal scaling, high-write throughput (social media feeds, logs, real-time analytics).

Q: What is an index and why does it matter?

A: An index is a separate data structure (B-Tree) that stores pointers to rows sorted by a column. Speeds up SELECT by avoiding full table scan. Slows INSERT/UPDATE because index must be updated.

Q: Explain ACID properties.

A: Atomicity: all-or-nothing transaction. Consistency: DB always goes from valid state to valid state. Isolation: concurrent transactions don't interfere. Durability: committed data survives crashes.

■ 7. Git & Version Control — The Recipe Book

Command	Purpose
git init / clone	Initialise or clone a repository.
git add . / git commit -m	Stage and commit changes.
git push / pull	Sync with remote.
git branch -b feature	Create new branch.
git merge / rebase	Merge: creates merge commit. Rebase: replays commits on top for linear history.
git stash	Temporarily shelve uncommitted changes.
git log --oneline --graph	Visual commit history.
git reset HEAD~1	Undo last commit (soft=keep changes staged).
git revert <hash>	Create new commit that undoes a past commit (safe for shared branches).
git cherry-pick <hash>	Apply a specific commit to current branch.

Q: Merge vs Rebase?

A: Merge creates a merge commit preserving branch history. Rebase rewrites commit history for a cleaner linear log. Don't rebase public/shared branches.

Q: What is a pull request?

A: A request to merge a feature branch into the main branch. Enables code review, discussion, CI checks before merging.

Q: How do you resolve a merge conflict?

A: Git marks conflicting areas with <<<<<<, =====, >>>>>>. Manually edit to resolve, then git add and git commit.

■ 8. AI/ML & Syslab.ai Stack — The Secret Sauce

Syslab.ai focuses on Deep Learning, NLP, Computer Vision, Generative AI, IoT, and Blockchain. As a full-stack developer here you must understand how to integrate AI models into web applications.

Deep Learning & NLP Basics

- **Neural Network** — Layers of neurons (input → hidden → output). Learns via backpropagation + gradient descent.
- **Transformer** — Architecture behind GPT, BERT. Self-attention mechanism. Foundation of modern NLP.
- **LLM (Large Language Model)** — e.g., GPT-4, Claude. Trained on massive text data. Used in chatbots, code generation.
- **Embeddings** — Dense vector representations of text/images. Similar meaning → similar vectors.
- **Fine-tuning** — Training a pre-trained model further on domain-specific data.
- **RAG (Retrieval-Augmented Generation)** — LLM + vector database. Retrieve relevant docs, inject into prompt.

Computer Vision

- **CNN (Convolutional Neural Network)** — Feature extraction from images via conv layers.
- **YOLO** — Real-time object detection. Used in Syslab's Eyecon AI for smart city event detection.
- **Vision Transformers (ViT)** — Transformer architecture applied to image patches.
- **OpenCV** — Python library for image processing (resize, threshold, edge detection).

AI Integration in Full-Stack

- **REST API wrapping models** — FastAPI/Flask serving ML model predictions as endpoints.
- **WebSocket** — Real-time streaming responses (like ChatGPT). ws:// protocol.
- **Async processing** — Long-running inference jobs handled via message queues (Redis, Celery).
- **Vector DB** — Pinecone, Weaviate, pgvector. Store embeddings for semantic search / RAG.

Syslab.ai Products — Know These

- **ProctorParhai** — AI online proctoring. Face detection, gaze tracking, unauthorized object detection. Computer Vision + ML.
- **iParhai** — Adaptive learning platform. NLP-driven content delivery personalised to learner needs.
- **Eyecon AI** — Smart city event detection using Vision Transformers + Visual Language Models.
- **Recruitment System (uHire)** — AI-driven candidate evaluation and screening for UBL.

Q: How would you integrate an ML model into a web application?

A: 1) Wrap model in FastAPI endpoint returning JSON predictions. 2) Store model artefacts (ONNX/pickle). 3) Load model once on startup, reuse session. 4) Queue heavy inference with Celery+Redis. 5) Return results via REST or WebSocket.

Q: What is a Vector Database?

A: A database optimised for storing and querying high-dimensional embedding vectors. Used for semantic search, RAG, and recommendation. Similarity search via cosine or dot-product distance (e.g., Pinecone, pgvector).

■ ■ 9. System Design — The Chef's Blueprint

- **Scalability** — Horizontal (more servers) vs Vertical (bigger server). Prefer horizontal.
- **Load Balancer** — Distributes traffic across server instances (Nginx, AWS ALB). Round-robin or least-connections.
- **Caching** — Redis for session, computed results, DB query cache. CDN for static assets. Cache-aside pattern.
- **Microservices** — Small independent services communicating via REST or message queues. Pros: independent deploy. Cons: network overhead, complexity.
- **Message Queues** — RabbitMQ / Kafka / Redis Streams. Decouple producers from consumers. Buffer spikes.
- **Database Sharding** — Horizontal partitioning of data across multiple DB instances.
- **CDN** — Content Delivery Network. Serve static assets from edge nodes closest to user (Cloudflare, AWS CloudFront).
- **Rate Limiting** — Prevent abuse. Token bucket or sliding window algorithm.
- **WebSockets** — Persistent bidirectional connection. Used for real-time features (chat, live proctoring).
- **Docker & Kubernetes** — Docker: containerise apps. K8s: orchestrate containers (scaling, self-healing, rolling updates).
- **CI/CD** — Automated build, test, deploy pipeline. GitHub Actions / Jenkins / GitLab CI.

Q: How would you design a real-time proctoring system like ProctorParhai?

A: WebRTC/WebSocket for live video stream. Computer Vision model (face detection, gaze, object detection) running on backend workers. Results streamed back via WebSocket. Redis for session state. Message queue for async event processing. Object storage (S3) for session recordings.

Q: How do you handle authentication at scale?

A: Stateless JWT tokens (no server-side session storage). Refresh token rotation. Store refresh tokens in HTTP-only cookies. Use Redis for token blacklisting on logout. Rate limit auth endpoints.

■ 10. Aptitude & Logic — The Taste Test

Data Structures — Quick Reference

Structure	Key Points
Array	O(1) access, O(n) search/insert. Contiguous memory.
Linked List	O(n) access, O(1) insert/delete at head. Non-contiguous.
Stack	LIFO. push() / pop() — O(1). Used for: undo, brackets, DFS.
Queue	FIFO. enqueue/dequeue — O(1) (with linked list). BFS, task scheduling.
Hash Map	Average O(1) insert/search. Collisions handled via chaining/open addressing.
Binary Tree	Hierarchical. BST: O(log n) search if balanced.
Graph	Nodes + edges. DFS (stack/recursion), BFS (queue). Dijkstra for shortest path.
Heap	Complete binary tree. Min-heap: parent ≤ children. O(log n) insert/extract. Priority queues.

Big-O Cheat Sheet

- O(1) — Constant: hash map lookup, array index.
- O(log n) — Logarithmic: binary search, balanced BST operations.
- O(n) — Linear: single loop, linear search.
- O(n log n) — Merge sort, heap sort.
- O(n²) — Bubble/insertion/selection sort; nested loops.
- O(2ⁿ) — Recursive Fibonacci without memoization.

Common Aptitude Patterns

- **Two Pointers** — Start pointers at both ends, move inward. E.g., check palindrome, pair sum in sorted array.
- **Sliding Window** — Expand/shrink window. Max subarray sum of length k.
- **Binary Search** — Sorted array. Halve search space each step.
- **Dynamic Programming** — Break into overlapping subproblems. Memoize results.
- **BFS/DFS** — BFS: shortest path in unweighted graph. DFS: connected components, topological sort.

11. Code Snippets Bank — The Cookbook

Debounce Function (JavaScript)

Returns a debounced version of a function that delays invoking fn until after wait ms have elapsed since the last call.

● JavaScript

```
function debounce(fn, wait) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), wait);
  };
}

// Usage
const handleSearch = debounce((e) => {
  console.log('Searching:', e.target.value);
}, 300);
input.addEventListener('input', handleSearch);
```

Flatten Nested Array (JavaScript)

● JavaScript

```
// Recursive approach
function flattenDeep(arr) {
  return arr.reduce((acc, val) =>
    Array.isArray(val)
      ? acc.concat(flattenDeep(val))
      : acc.concat(val),
    []);
}

// ES2019 native
const flat = [1, [2, [3, [4]]]].flat(Infinity);
// [1, 2, 3, 4]
```

Custom Hook — useFetch (React)

● React / JavaScript

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    fetch(url)
      .then(r => r.json())
      .then(d => { if (!cancelled) { setData(d); setLoading(false); }})
      .catch(e => { if (!cancelled) { setError(e); setLoading(false); }});
    return () => { cancelled = true; }; // cleanup
  }, [url]);

  return { data, loading, error };
}

// Usage
const { data, loading } = useFetch('/api/users');
```

JWT Auth Middleware (Node.js / Express)

● Node.js / Express

```
const jwt = require('jsonwebtoken');

function authMiddleware(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer

  if (!token) return res.status(401).json({ error: 'No token' });

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.status(403).json({ error: 'Invalid token' });
    req.user = user;
    next();
  });
}

// Protect routes
app.get('/api/profile', authMiddleware, (req, res) => {
  res.json({ user: req.user });
});
```

SQL — Top N per Group

● SQL

```
-- Top 3 highest-paid employees per department
SELECT department, name, salary
FROM (
  SELECT
    department,
    name,
    salary,
    ROW_NUMBER() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS rn
  FROM employees
) ranked
WHERE rn <= 3;
```

MongoDB — Aggregation Pipeline

● MongoDB

```
// Average score per category, sorted descending
db.results.aggregate([
  { $match: { status: 'completed' } },
  { $group: {
    _id: '$category',
    avgScore: { $avg: '$score' },
    count: { $sum: 1 }
  }},
  { $sort: { avgScore: -1 } },
  { $project: { category: '$_id', avgScore: 1, count: 1, _id: 0 } }
]);
```


Binary Search (Python)

● Python

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# O(log n) time, O(1) space
print(binary_search([1,3,5,7,9,11], 7)) # -> 3
```

Two-Sum (Sorted Array) — Two Pointers (Python)

● Python

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1
        else:
            right -= 1
    return []

# O(n) time, O(1) space
print(two_sum_sorted([2,7,11,15], 9)) # -> [0, 1]
```

Max Sum Subarray of Size K — Sliding Window (Python)

● Python

```
def max_sum_subarray(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum

# O(n) time
print(max_sum_subarray([2,1,5,1,3,2], 3)) # -> 9
```

FastAPI — ML Model Endpoint (Python)

● Python / FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
import numpy as np

app = FastAPI()

# Simulated model load (replace with real model)
class FakeModel:
    def predict(self, data): return [float(sum(data))]

model = FakeModel()

class PredictRequest(BaseModel):
    features: list[float]

class PredictResponse(BaseModel):
    prediction: list[float]

@app.post('/predict', response_model=PredictResponse)
async def predict(req: PredictRequest):
    result = model.predict(req.features)
    return PredictResponse(prediction=result)

# Run: uvicorn main:app --reload
```

Reverse a Linked List (Python)

● Python

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

def reverse_linked_list(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev # new head

# O(n) time, O(1) space
```

■ Quick Interview Tips — Last Sip

Think Aloud	Narrate your reasoning. Interviewers want to see how you think, not just the answer.
Start Brute Force	Mention the naive $O(n^2)$ approach first, then optimise. Shows structured thinking.
Edge Cases	Always ask: empty input? null? negative numbers? duplicates?
Time & Space	State complexity of every solution: $O(n)$ time, $O(1)$ space.
Syslab Context	Relate answers to AI/ML integration. Show awareness of their products (ProctorParhai, Eyecon AI).
Clean Code	Meaningful variable names, single responsibility, comments on non-obvious logic.
Questions to Ask	"What does your ML deployment pipeline look like?" / "How do you handle model versioning?"

Good luck! ■ Brew confidence, serve excellence.